

```
# prompt: Install sympy
# numpy and drawsvg

!pip install sympy numpy drawsvg

Requirement already satisfied: sympy in
/usr/local/lib/python3.10/dist-packages (1.13.1)
Requirement already satisfied: numpy in
/usr/local/lib/python3.10/dist-packages (1.26.4)
Collecting drawsvg
  Downloading drawsvg-2.4.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.10/dist-packages (from sympy) (1.3.0)
Downloading drawsvg-2.4.0-py3-none-any.whl (44 kB)
_____ 44.1/44.1 kB 1.9 MB/s eta
0:00:00
```

Light Guide Panel Generator

Light Guide Panels are the (literal) backbone of LCD screen illumination. They're the diffusion secret sauce behind uniformly backlit LCD displays. When illuminated from the edge, they illuminate outwards through the pattern etched on the surface of therylic.

It turns out that design software for this sort of thing [already exists](#). But why pay \$1500 when we can write a crude version that will do the job?

```
#!/usr/bin/env python3
#%matplotlib inline
import matplotlib.pyplot as plt
import drawsvg as draw
import numpy as np
from scipy.optimize import curve_fit
import scipy.interpolate as interpolate
from ipywidgets import interactive
from IPython.display import SVG, display, HTML
import base64

# Extra fn for rescaling displayed SVGs
_html_template=''

# def svg_to_fixed_width_html_image(svg, width="100%"):
#     text = _html_template.format(width, base64.b64encode(svg))
#     return HTML(text)

def svg_to_fixed_width_html_image(svg, width="50%"):
    b64 = base64.b64encode(svg).decode("utf8")
    text = f''
    return HTML(text)

R_OUTER = 15
R_INNER = 5

HALF_WIDTH_MM = (10)/ 2
HEIGHT_MM = 345.5

HALF_WIDTH_MM = (R_OUTER - R_INNER) / 2
HEIGHT_MM = (R_OUTER + R_INNER) * np.pi
NOMINAL_DOTS_PER_MM = 1.0
DOT_DIAMETER_MM = 0.1
CORNER_RADIUS_MM = 0

HEIGHT_MM = round(HEIGHT_MM, 1)
print(f"Half Width: {HALF_WIDTH_MM}")
print(f"Height: {HEIGHT_MM}")

Half Width: 5.0
Height: 62.8

```

We can create a spline function with handles that can help us model dot density as a function of distance from the light source.

```

def spline_density_gen(y0, x1, y1, x2, y2, x3, y3, y4):
    x = np.array([0, x1, x2, x3, HALF_WIDTH_MM]) # x axis control
    points
    y = np.array([y0, y1, y2, y3, y4]) # y control points.
    t, c, k = interpolate.splrep(x, y, s=0, k=2)
    spline = interpolate.BSpline(t, c, k, extrapolate=False)
    # plot:
    plt.figure(figsize=(10, 5))
    #plt.axes().set_aspect(100)#('equal')
    plt.plot(x, y, 'bo', label='Control Points') # Plot the control
    points.
    for i,xy in enumerate(zip(x, y)):
        plt.annotate(f"p{i}", xy=xy)
    N = 100 # Plot resolution.
    xx = np.linspace(x.min(), x.max(), N)
    yy = spline(xx)
    plt.plot(xx, yy, 'r', label='BSpline')
    plt.plot(xx, [1/NOMINAL_DOTS_PER_MM]*len(xx))
    plt.grid()
    plt.legend(loc='best')
    plt.title("Half Length Illumination Density")
    plt.ylabel("dot density [dots/mm]")
    plt.xlabel("distance from center [mm]")
    plt.tight_layout()
    #plt.ylim(0,1)

```

```

print([y0, x1, y1, x2, y2, x3, y3, y4])
plt.show()
return t,c,k

i_sdf = interactive(spline_density_gen,
                    y0=(0, 1/NOMINAL_DOTS_PER_MM*2, 0.01),
                    x1=(0, HALF_WIDTH_MM,0.25),
                    y1=(0, 1/NOMINAL_DOTS_PER_MM*2, 0.01),
                    x2=(0,HALF_WIDTH_MM,0.25),
                    y2=(0, 1/NOMINAL_DOTS_PER_MM*2, 0.01),
                    x3=(0,HALF_WIDTH_MM,0.25),
                    y3=(0, 1/NOMINAL_DOTS_PER_MM*2, 0.01),
                    y4=(0, 1/NOMINAL_DOTS_PER_MM*2, 0.01))

display(i_sdf)

{"model_id":"6c7c21984a064610ae5d52fae82aa930","version_major":2,"version_minor":0}

dot_step_factor = 1 # how far apart dots should be

# Create the spline function from the density parameters above.
rho = interpolate.BSpline(*i_sdf.result, extrapolate=False)
def dotspace(x0, xf, endpoint=False):
    """return an array of dot locations from x0 to xf spaced
    according to the dot density profile."""
    dots_x = []
    x = x0
    while x < xf:
        dots_x.append(x)
        dots_per_mm_x = rho(x) / dot_step_factor
        print(x, dots_per_mm_x)
        x += 1/dots_per_mm_x # Step forward mm-per-dot
    return np.array(dots_x)

Y = np.arange(0, HEIGHT_MM, NOMINAL_DOTS_PER_MM*2)[: -1]
X = dotspace(0, HALF_WIDTH_MM - NOMINAL_DOTS_PER_MM)
xx, yy = np.meshgrid(X, Y) # create HALF of the x and y index of all
the points given their x and y separately in rows and columns

print(X)
print(dotspace(0, HALF_WIDTH_MM - NOMINAL_DOTS_PER_MM))
yoffset = (Y[1] - Y[0])/2 # to make the lines centered, because we set
the canvas origin horizontally centered, to we only need to calculate
the top offset
xoffset = (X[1] - X[0])/2 # to make the alternate lines at the center
of the other lines
Y2 = np.arange(yoffset, HEIGHT_MM+yoffset, NOMINAL_DOTS_PER_MM*2)[: -1]
X2 = dotspace(xoffset, HALF_WIDTH_MM - NOMINAL_DOTS_PER_MM)
xx2, yy2 = np.meshgrid(X2,Y2) # this creates the other, offsetted HALF

```

```

# Move non-illuminated sides off the top/bottom edge.
miny = min(yy[0][0], yy2[0][0])
maxy = max(yy[-1][-1], yy2[-1][-1])
deltay = HEIGHT_MM - (maxy-miny)
yy += deltay/2
yy2 += deltay/2

# TODO: we need to renormalize the output since it has been stretched
beyond the width of the original img.

# Preview Plot for sanity checking:
# fig = plt.figure(figsize=(15,15))
# ax = fig.add_subplot(111)
# ax.set_aspect('equal')
# ax.plot(xx, yy, ls="None", marker=".", color="blue")
# ax.plot(xx2, yy2, ls="None", marker=".", color="green")
# ax.plot(-xx, yy, ls="None", marker=".", color="blue")
# ax.plot(-xx2, yy2, ls="None", marker=".", color="green")
# plt.tight_layout()
# plt.show()

0 1.630000000000000003
0.6134969325153373 1.6562565202613524
1.2172681405177082 1.7071173586770314
1.803050948546192 1.7801853097210392
2.3647902205979894 1.8706063423423842
2.8993762408441275 1.9287877608349884
3.4178366024792988 1.9588313197826566
3.9283450826637774 1.980430149493131
[0. 0.61349693 1.21726814 1.80305095 2.36479022 2.89937624
 3.4178366 3.92834508]
0 1.630000000000000003
0.6134969325153373 1.6562565202613524
1.2172681405177082 1.7071173586770314
1.803050948546192 1.7801853097210392
2.3647902205979894 1.8706063423423842
2.8993762408441275 1.9287877608349884
3.4178366024792988 1.9588313197826566
3.9283450826637774 1.980430149493131
[0. 0.61349693 1.21726814 1.80305095 2.36479022 2.89937624
 3.4178366 3.92834508]
0.30674846625766866 1.6399249190172882
0.9165324803936585 1.6786812726731828
1.512238178836085 1.740990298385274
2.0866239183358832 1.8239496269883158
2.634884673569123 1.9061835709251997
3.1594931190792437 1.944882280183103
3.6736630563496133 1.9706452187053938

```

Draw the SVG from the above parameters.

```
def translate_to_polar_offset(x, y, y_max, xoffset=0):
    # y might be negative $$
    x = x + xoffset
    y_temp = (np.pi / y_max) * y
    # x' = r * cos(theta)
    polar_x = x * np.cos(y_temp)
    # y' = r * sin(theta)
    polar_y = x * np.sin(y_temp)
    return polar_x, polar_y

def prepare_points(x, y):
    # x_new = x + square_width / 2
    # y_new = y - square_height / 2
    x_new = x + HALF_WIDTH_MM
    y_new = y - HEIGHT_MM / 2
    return x_new, y_new

# Create an SVG replacing the two meshgrids from above with actual
# circles.
offset = (10, 10)
fill_color = 'black'

# d = draw.Drawing(2*HALF_WIDTH_MM + 2*offset[0],
#                 HEIGHT_MM + 2*offset[0],
#                 origin=(-HALF_WIDTH_MM - offset[0], 0),
#                 displayInline=False)
# d.set_pixel_scale(1)
# r = draw.Rectangle(-HALF_WIDTH_MM, 0,
#                   2*HALF_WIDTH_MM, HEIGHT_MM,
#                   rx=CORNER_RADIUS_MM,
#                   fill='#FFFFFF', stroke='red', stroke_width="0.1")
# d.append(r)
# # Produce the circle pattern.
# # Skip duplicate circles to avoid weird bug where LaserCAD software
# # skips etching anything
# # if there are copies at the same location.
# #for xx, yy, fill_color in [[xx, yy, 'red'], [xx2, yy2, 'black'], [-
# xx[:,1:], yy[:,1:], 'green'], [-xx2, yy2, 'black']]:
# for xx, yy in [[xx, yy], [xx2, yy2], [-xx[:,1:], yy[:,1:]], [-xx2,
# yy2]]:
#     for x,y in zip(xx.flatten(), yy.flatten()):
#         c = draw.Circle(x, y, r=DOT_DIAMETER_MM/2.,
#                         stroke_width='0.1', stroke=fill_color,
#                         fill_opacity=1.0)#, id='circle')
#         d.append(c)
# d.save_svg('example.svg')

d3 = draw.Drawing(2*HALF_WIDTH_MM + 2*offset[0],
```

```

        HEIGHT_MM + 2*offset[1],
        origin=(0, -HEIGHT_MM / 2 - offset[1]),
        displayInline=False)
d3.set_pixel_scale(1)
r = draw.Rectangle(0, -HEIGHT_MM / 2,
                   2*HALF_WIDTH_MM, HEIGHT_MM,
                   rx=CORNER_RADIUS_MM,
                   fill='FFFFFF', stroke='red', stroke_width="0.1")
d3.append(r)

# Produce the circle pattern.
# Skip duplicate circles to avoid weird bug where LaserCAD software
# skips etching anything
# if there are copies at the same location.
#for xx, yy, fill_color in [[xx, yy, 'red'], [xx2, yy2, 'black'], [-
xx[:,1:], yy[:,1:], 'green'], [-xx2, yy2, 'black']]:
    for xx, yy in [[xx, yy], [xx2, yy2], [-xx[:,1:], yy[:,1:]], [-xx2,
yy2]]:
        for x,y in zip(xx.flatten(), yy.flatten()):
            x, y = prepare_points(x, y)
            delta_left = random.uniform(DOT_DIAMETER_MM / 4,
DOT_DIAMETER_MM / 2)
            delta_right = random.uniform(DOT_DIAMETER_MM / 4,
DOT_DIAMETER_MM / 2)
            # 3 points:
            # center point (x, y)
            # bottom point (x, y - d_left)
            # top point (x, y + d_right)
            c = draw.Line(x, y - delta_left, x, y + delta_right,
stroke=fill_color, stroke_width="0.05")
            d3.append(c)

x_axis = draw.Line(0, 0, 2*HALF_WIDTH_MM, 0, stroke='black',
stroke_width="0.1")
d3.append(x_axis)
print(HEIGHT_MM)
y_axis = draw.Line(0, 0, 0, HEIGHT_MM / 2, stroke='blue',
stroke_width="0.1")
d3.append(y_axis)

d3.save_svg('example_moved.svg')

62.8

#display(SVG(url='https://upload.wikimedia.org/wikipedia/commons/f/
f6/People_%28example%29.svg'))
svg_to_fixed_width_html_image(SVG('example_moved.svg').data.encode('as
cii'), width="100%")

```

<IPython.core.display.HTML object>

```
d2 = draw.Drawing(2 * R_OUTER + 2 * offset[0],
                  2 * R_OUTER + 2 * offset[1],
                  origin = (-R_OUTER - offset[0], -R_OUTER -
offset[1]),
                  displayInline=False)
d2.set_pixel_scale(1)
r2 = draw.Circle(0, 0, r=R_OUTER, fill='#FFFFFF', stroke='red',
stroke_width="0.1")
d2.append(r2)
r3 = draw.Circle(0, 0, r=R_INNER, fill='#FFFFFF', stroke='red',
stroke_width="0.1")
d2.append(r3)

for xx, yy in [[xx, yy], [xx2, yy2], [-xx[:,1:], yy[:,1:]], [-xx2,
yy2]]:
    for x,y in zip(xx.flatten(), yy.flatten()):
        x, y = prepare_points(x, y)

        delta_left = random.uniform(DOT_DIAMETER_MM * 1,
DOT_DIAMETER_MM * 2)
        delta_right = random.uniform(DOT_DIAMETER_MM * 1,
DOT_DIAMETER_MM * 2)

        polar_line_x, polar_line_y = translate_to_polar_offset(x, y +
delta_left, HEIGHT_MM / 2, R_INNER)

        polar_end_x, polar_end_y, = translate_to_polar_offset(x, y -
delta_right, HEIGHT_MM / 2, R_INNER)

        c = draw.Line(polar_line_x, polar_line_y,
                      polar_end_x, polar_end_y,
                      stroke=fill_color, stroke_width="0.05")
        # print(x, y, polar_line_x)
        d2.append(c)

d2.save_svg("example_polar.svg")

#display(SVG(url='https://upload.wikimedia.org/wikipedia/commons/f/
f6/People_%28example%29.svg'))
svg_to_fixed_width_html_image(SVG('example_polar.svg').data.encode('as
cii'), width="100%")
```

<IPython.core.display.HTML object>

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import UnivariateSpline

def visualize_spline_with_randomized_dots_2d(spline_func, x_range,
```

```

y_range, num_x_areas, num_y_areas, num_points_total):
    """
    Visualize a spline equation and overlay randomized dots with
    density proportional to y values.
    The plot is divided into areas along both x and y axes.

    Parameters:
    - spline_func: The spline function (callable) to visualize.
    - x_range: Tuple (x_min, x_max) defining the range of x values.
    - y_range: Tuple (y_min, y_max) defining the range of y values.
    - num_x_areas: Number of areas to divide the x-axis into.
    - num_y_areas: Number of areas to divide the y-axis into.
    - num_points_total: Total number of dots to distribute across all
    areas.
    """
    # Generate x values for the spline
    x = np.linspace(x_range[0], x_range[1], 1000)
    y = spline_func(x)

    # Normalize y to use as weights
    y_normalized = (y - np.min(y)) / (np.max(y) - np.min(y)) # Scale
y values to [0, 1]
    y_weights = y_normalized / np.sum(y_normalized) # Normalize
weights

    # Define the boundaries of the areas
    x_boundaries = np.linspace(x_range[0], x_range[1], num_x_areas +
1)
    y_boundaries = np.linspace(y_range[0], y_range[1], num_y_areas +
1)

    # Create the plot
    plt.figure(figsize=(10, 6))
    plt.plot(x, y, label='Spline Equation', color='blue', lw=2)

    dots = [] # List to store the dots' positions

    # Distribute dots across the areas
    base_dots_per_area = 0
    for i in range(num_x_areas):
        for j in range(num_y_areas):
            # Define area boundaries
            x_min, x_max = x_boundaries[i], x_boundaries[i + 1]
            y_min, y_max = y_boundaries[j], y_boundaries[j + 1]

            # Find x values within the current x area
            mask_x = (x >= x_min) & (x < x_max)
            # print(mask_x)
            x_in_area = x[mask_x]
            y_in_area = y[mask_x]

```

```

        # Calculate area weight based on y values
        if len(y_in_area) > 0:
            area_weight = np.sum(y_weights[mask_x])
        else:
            area_weight = 0

        # print(area_weight)

        # Calculate the number of dots in this area
        weighted_dots = int(num_points_total * area_weight)
        # print(weighted_dots)
        num_dots = max(base_dots_per_area, weighted_dots) #
        Ensure at least the base number of dots

        # Generate random positions for the dots within this area
        x_dots = np.random.uniform(x_min, x_max, num_dots)
        y_dots = np.random.uniform(y_min, y_max, num_dots)

        # Add the dots to the list
        dots.extend(zip(x_dots, y_dots))

        # Plot the dots
        plt.scatter(x_dots, y_dots, color='red', s=10, alpha=0.6,
label=None)

    # Finalize the plot
    plt.title('Spline Visualization with Randomized Dots')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.legend()
    plt.grid(True)
    plt.show()

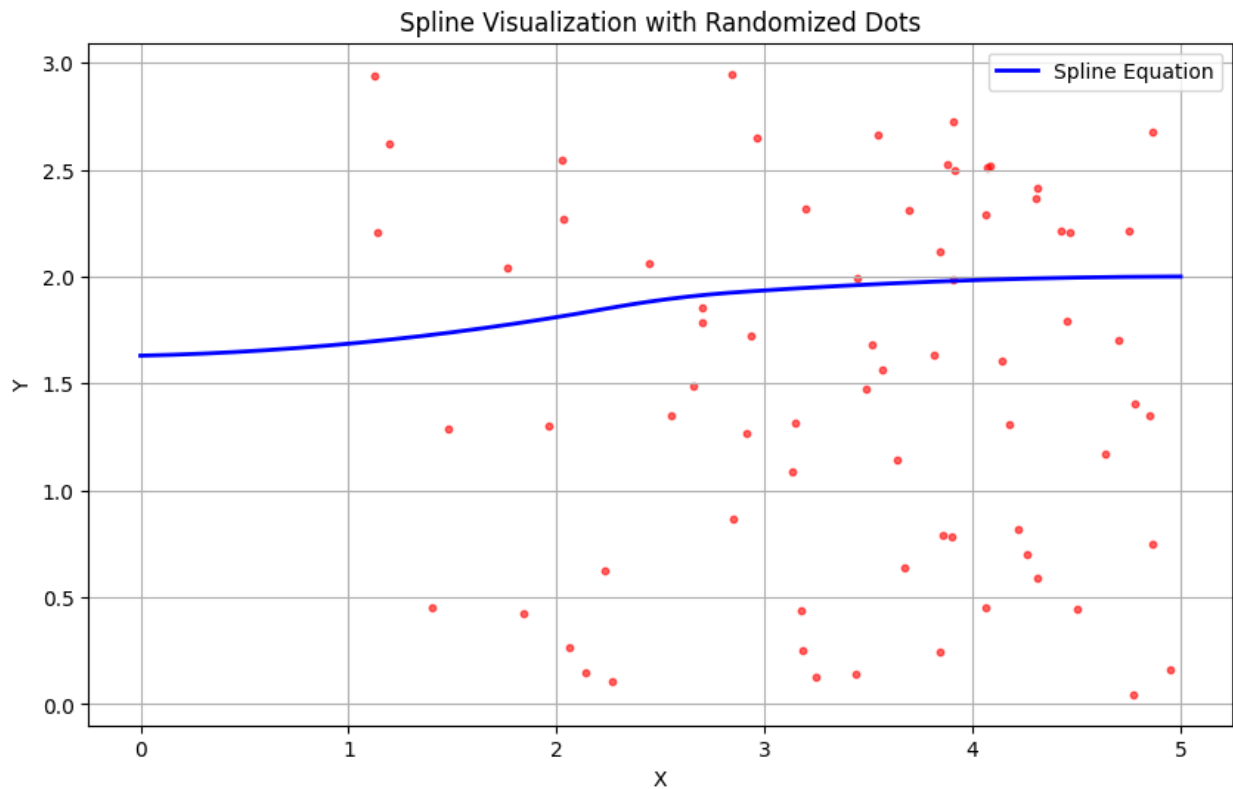
    return dots

# Example Usage
# Define a spline function
x = np.linspace(0, 10, 10)
y = np.sin(x) + 0.5 * x
spline_func = UnivariateSpline(x, y, s=1)

# Visualize the spline with randomized dots
engrave_dots = visualize_spline_with_randomized_dots_2d(
    rho,
    x_range=(0, 5),
    y_range=(0, 3),
    num_x_areas=5,
    num_y_areas=4,
    num_points_total=20

```

```
)
print(engrave_dots)
```



```
[ (1.4045670529192789, 0.4529428709154112), (1.8425155630172985,
0.42670384379711035), (1.9636072333733239, 1.3044477496141846),
(1.4824020428294125, 1.290806907411995), (1.1391237056593446,
2.206371008137375), (1.763098607380439, 2.0431379200617963),
(1.193229814495659, 2.6228840792439265), (1.121481075474756,
2.9366289642547603), (2.136392035806433, 0.14787769685162996),
(2.062580895973504, 0.2631579262361553), (2.2646523933492526,
0.10632870619463147), (2.229304103629031, 0.6274261062831419),
(2.8471829214375637, 0.8669692314075077), (2.5512683055956513,
1.3507267756559114), (2.910502463319362, 1.2704557029759953),
(2.655907995873761, 1.4890828770203648), (2.702010377732515,
1.7882490639630468), (2.935355085464092, 1.7203717027386747),
(2.703537824971, 1.8546073761720314), (2.443331522214644,
2.0604284716720707), (2.0349650648616486, 2.267590037694786),
(2.024882258869397, 2.5450149946848812), (2.8390622702289416,
2.947614673239333), (2.963428937096344, 2.645686967194113),
(3.1809354997813593, 0.25466117439993147), (3.840655136141875,
0.24550958521441055), (3.669482934417558, 0.6416736110774619),
(3.1761183647181626, 0.4393747943779352), (3.2476395685799444,
0.12733325603196896), (3.4379700528601758, 0.1398259322514523),
```

```

(3.1302281615485903, 1.0867582675906629), (3.637768066688101,
1.1438927542669632), (3.9020604179525895, 0.7806675573202025),
(3.144217153111968, 1.3171161599553534), (3.8559720634143395,
0.7894348786418139), (3.487676240452426, 1.4727245811965064),
(3.841378874974126, 2.1201106109794248), (3.562699207101339,
1.562123581198407), (3.44757016276653, 1.9897424116725482),
(3.5165464453459103, 1.6839354328895955), (3.907217196090198,
1.9874487171495512), (3.814867163695466, 1.631116385982033),
(3.9163341985779487, 2.4954978880693965), (3.8755830155392728,
2.5267497802505234), (3.5469735345287314, 2.662638669523634),
(3.1987143304482504, 2.3168996643248683), (3.695778611893515,
2.3101869493428913), (3.909656783075274, 2.7234496790521203),
(4.259775013154966, 0.7014441439177155), (4.061772051129165,
0.4547094377586283), (4.947824015890163, 0.15899852202882495),
(4.314749666386317, 0.5900379081763262), (4.774171712867254,
0.04670620948020071), (4.501175472689331, 0.44862915673437254),
(4.861731574401896, 0.7524565590865705), (4.84758660243587,
1.3507831145216749), (4.17661603147963, 1.3090785690752664),
(4.776523510920833, 1.4075609146915258), (4.635735283713128,
1.1686810835454293), (4.219662784770366, 0.8214937994484828),
(4.705112538630566, 1.7027632295959017), (4.753425224713821,
2.2141573853709957), (4.425513115563333, 2.210364028832699),
(4.138092549008151, 1.6037636824403818), (4.454223634885775,
1.7925956546815303), (4.46620042708826, 2.2079932911621967),
(4.082998699534887, 2.520887190123424), (4.309718344699552,
2.4119496953797617), (4.069998634592265, 2.511087720552394),
(4.863098054758562, 2.676095971673503), (4.064982869214185,
2.2926865442579194), (4.301560473548359, 2.3659680547024364)]

def scale_and_move_points(points, from_x, from_y, to_x, to_y):
    scaled_points = []

    x_scale = to_x / from_x
    y_scale = to_y / from_y

    for x, y in points:
        new_x = x * x_scale
        new_y = y * y_scale - to_y / 2 # Move down by y_scale / 2
        scaled_points.append((new_x, new_y))
    return scaled_points

enlarged_dots = scale_and_move_points(engrave_dots, 5, 3, 10, 10)

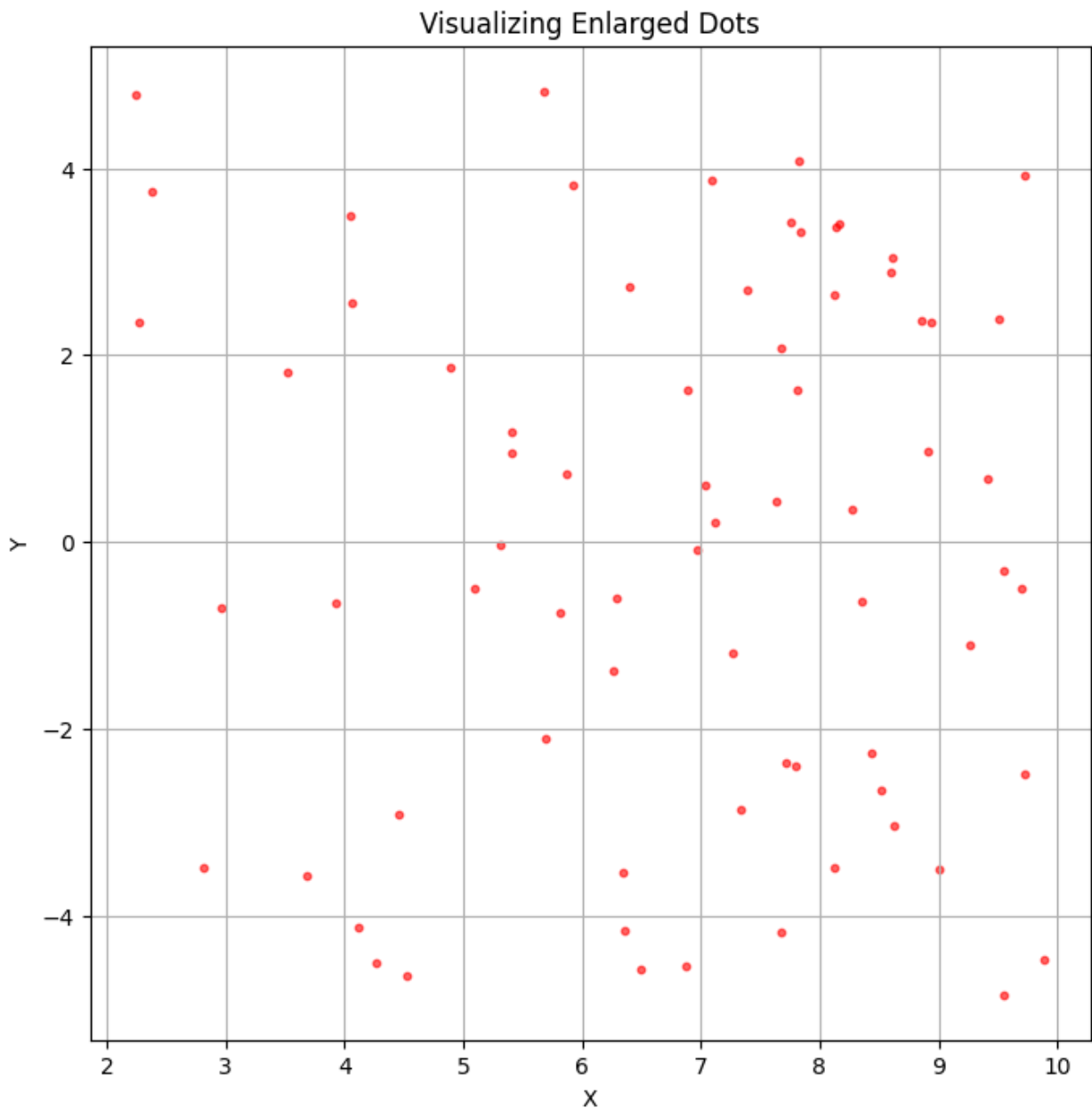
import matplotlib.pyplot as plt
import random

# Assuming 'enlarged_dots' is already defined from the previous code

plt.figure(figsize=(8, 8))
x_coords, y_coords = zip(*enlarged_dots)

```

```
plt.scatter(x_coors, y_coors, color='red', s=10, alpha=0.6) #  
Adjust 's' for dot size  
plt.title('Visualizing Enlarged Dots')  
plt.xlabel('X')  
plt.ylabel('Y')  
plt.grid(True)  
plt.show()
```



```
ENGRAVE_COLOR = "#00FF00"
```

```

import matplotlib.pyplot as plt
import drawsvg as draw
import numpy as np
from scipy.optimize import curve_fit
import scipy.interpolate as interpolate
from ipywidgets import interactive
from IPython.display import SVG, display, HTML
import base64
from scipy.interpolate import UnivariateSpline
import random

def translate_to_polar_offset(x, y, y_max, xoffset=0):
    x = x + xoffset
    y_temp = (np.pi / y_max) * y
    # x' = r * cos(theta)
    polar_x = x * np.cos(y_temp)
    # y' = r * sin(theta)
    polar_y = x * np.sin(y_temp)
    return polar_x, polar_y

# Now use enlarged_dots with translate_to_polar_offset in the drawing
d2 = draw.Drawing(2 * R_OUTER + 2 * offset[0],
                  2 * R_OUTER + 2 * offset[1],
                  origin=(-R_OUTER - offset[0], -R_OUTER - offset[1]),
                  displayInline=False)
d2.set_pixel_scale(1)
r2 = draw.Circle(0, 0, r=R_OUTER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r2)
r3 = draw.Circle(0, 0, r=R_INNER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r3)

for x, y in enlarged_dots:
    polar_x, polar_y = translate_to_polar_offset(x, y, 5, R_INNER) #
    Assuming 10 is the relevant y_max here, adjust if necessary
    c = draw.Circle(polar_x, polar_y, r=DOT_DIAMETER_MM/2,
stroke_width='0.1', stroke=ENGRAVE_COLOR, fill_opacity=1.0)
    d2.append(c)

d2.save_svg("ring_dots.svg")
svg_to_fixed_width_html_image(SVG('ring_dots.svg').data.encode('ascii'
), width="100%")

<IPython.core.display.HTML object>

%%python3 -m pip install "drawsvg[all]"

```

```

Requirement already satisfied: drawsvg[all] in
/usr/local/lib/python3.10/dist-packages (2.4.0)
Requirement already satisfied: cairoSVG~=2.3 in
/usr/local/lib/python3.10/dist-packages (from drawsvg[all]) (2.7.1)
Requirement already satisfied: numpy~=1.16 in
/usr/local/lib/python3.10/dist-packages (from drawsvg[all]) (1.26.4)
Requirement already satisfied: imageio~=2.5 in
/usr/local/lib/python3.10/dist-packages (from drawsvg[all]) (2.36.0)
Requirement already satisfied: imageio-ffmpeg~=0.4 in
/usr/local/lib/python3.10/dist-packages (from drawsvg[all]) (0.5.1)
Requirement already satisfied: pwnkit~=1.0 in
/usr/local/lib/python3.10/dist-packages (from drawsvg[all]) (1.2.2)
Requirement already satisfied: cairocffi in
/usr/local/lib/python3.10/dist-packages (from cairoSVG~=2.3-
>drawsvg[all]) (1.7.1)
Requirement already satisfied: cssselect2 in
/usr/local/lib/python3.10/dist-packages (from cairoSVG~=2.3-
>drawsvg[all]) (0.7.0)
Requirement already satisfied: defusedxml in
/usr/local/lib/python3.10/dist-packages (from cairoSVG~=2.3-
>drawsvg[all]) (0.7.1)
Requirement already satisfied: pillow in
/usr/local/lib/python3.10/dist-packages (from cairoSVG~=2.3-
>drawsvg[all]) (11.0.0)
Requirement already satisfied: tinycss2 in
/usr/local/lib/python3.10/dist-packages (from cairoSVG~=2.3-
>drawsvg[all]) (1.4.0)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.10/dist-packages (from imageio-ffmpeg~=0.4-
>drawsvg[all]) (75.1.0)
Requirement already satisfied: cffi>=1.1.0 in
/usr/local/lib/python3.10/dist-packages (from cairocffi-
>cairoSVG~=2.3->drawsvg[all]) (1.17.1)
Requirement already satisfied: webencodings in
/usr/local/lib/python3.10/dist-packages (from cssselect2-
>cairoSVG~=2.3->drawsvg[all]) (0.5.1)
Requirement already satisfied: pycparser in
/usr/local/lib/python3.10/dist-packages (from cffi>=1.1.0->cairocffi-
>cairoSVG~=2.3->drawsvg[all]) (2.22)

```

```

import drawsvg as dw
import numpy as np

```

```

def calculate_arc_parameters(p1, p2, p3):

```

```

    """
    Calculate the SVG arc parameters to ensure the arc passes through
    three points.

```

```

    Parameters:

```

```

    - p1, p2, p3: tuples representing (x, y) coordinates of three
    points.

```

```

Returns:
- rx, ry: Radii of the ellipse
- cx, cy: Center of the ellipse
- rotation: Rotation of the ellipse (in degrees)
- large_arc_flag: 0 or 1
- sweep_flag: 0 or 1
"""
# Create a matrix for solving the circle's center
A = np.array([
    [p1[0] - p2[0], p1[1] - p2[1]],
    [p1[0] - p3[0], p1[1] - p3[1]]
])
b = np.array([
    (p1[0]**2 - p2[0]**2 + p1[1]**2 - p2[1]**2) / 2,
    (p1[0]**2 - p3[0]**2 + p1[1]**2 - p3[1]**2) / 2
])
center = np.linalg.solve(A, b)

# Compute the radius
radius = np.sqrt((p1[0] - center[0])**2 + (p1[1] - center[1])**2)

# Set arc parameters
rx = ry = radius
rotation = 0 # No rotation for a circle-based arc
large_arc_flag = 1
sweep_flag = 1

return rx, ry, center[0], center[1], rotation, large_arc_flag,
sweep_flag

def draw_arc_with_three_points(p1, p2, p3):
    """
    Draw an arc passing through three points using drawsvg.
    """
    rx, ry, cx, cy, rotation, large_arc_flag, sweep_flag =
    calculate_arc_parameters(p1, p2, p3)

    # Create a drawing canvas
    d = dw.Drawing(500, 500, origin='center')

    # Draw the three points
    d.append(dw.Circle(p1[0], p1[1], 2, fill='red'))
    d.append(dw.Circle(p2[0], p2[1], 2, fill='green'))
    d.append(dw.Circle(p3[0], p3[1], 2, fill='blue'))

    # Draw the arc
    d.append(dw.Path(f'M {p1[0]} {p1[1]} A {rx} {ry} {rotation}
    {large_arc_flag} {sweep_flag} {p3[0]} {p3[1]}',
        stroke='black', fill='none'))

```

```

        return d

# Example points
p1 = [50, 50]
p2 = [100, 100]
p3 = [150, 50]

# Draw and save the arc
d = draw_arc_with_three_points(p1, p2, p3)

svg_html = d.as_svg()
display(HTML(svg_html))

<IPython.core.display.HTML object>

# Now use enlarged_dots with translate_to_polar_offset in the drawing
d2 = draw.Drawing(2 * R_OUTER + 2 * offset[0],
                 2 * R_OUTER + 2 * offset[1],
                 origin=(-R_OUTER - offset[0], -R_OUTER - offset[1]),
                 displayInline=False)
d2.set_pixel_scale(1)
r2 = draw.Circle(0, 0, r=R_OUTER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r2)
r3 = draw.Circle(0, 0, r=R_INNER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r3)

for x, y in enlarged_dots:
    delta_y = random.uniform(DOT_DIAMETER_MM * 2, DOT_DIAMETER_MM * 5)
    delta_x = random.uniform(DOT_DIAMETER_MM * 1, DOT_DIAMETER_MM * 2)

    polar_x, polar_y = translate_to_polar_offset(x + delta_x, y, 5,
R_INNER)
    polar_x2, polar_y2 = translate_to_polar_offset(x, y - delta_y, 5,
R_INNER)
    polar_x3, polar_y3 = translate_to_polar_offset(x, y + delta_y, 5,
R_INNER)

    p = draw.Path(stroke=ENGRAVE_COLOR, fill='none', stroke_width=0.1)
    d2.append(p.M(polar_x2, polar_y2).Q(polar_x, polar_y, polar_x3,
polar_y3))
    d2.append(c)

d2.save_svg("example_polar_enlarged.svg")
svg_to_fixed_width_html_image(SVG('example_polar_enlarged.svg').data.e
ncode('ascii'), width="100%")

```

<IPython.core.display.HTML object>

```
# Now use enlarged_dots with translate_to_polar_offset in the drawing
d2 = draw.Drawing(2 * R_OUTER + 2 * offset[0],
                  2 * R_OUTER + 2 * offset[1],
                  origin=(-R_OUTER - offset[0], -R_OUTER - offset[1]),
                  displayInline=False)
d2.set_pixel_scale(1)
r2 = draw.Circle(0, 0, r=R_OUTER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r2)
r3 = draw.Circle(0, 0, r=R_INNER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r3)

for x, y in enlarged_dots:
    delta_y = random.uniform(DOT_DIAMETER_MM * 2, DOT_DIAMETER_MM * 5)

    all_points_on_line_x = np.full(10, x)
    all_points_on_line_y = np.linspace(y, y - delta_y, 10)

    print(all_points_on_line_x)
    print(all_points_on_line_y)

    all_points_polar_x = []
    all_points_polar_y = []

    for i in range(10):
        point_x = all_points_on_line_x[i]
        point_y = all_points_on_line_y[i]
        polar_x, polar_y = translate_to_polar_offset(point_x, point_y,
5, R_INNER)
        all_points_polar_x.append(polar_x)
        all_points_polar_y.append(polar_y)

    xy = [item for sublist in zip(all_points_polar_x,
all_points_polar_y) for item in sublist]
    d2.append(dw.Lines(*xy, stroke=ENGRAVE_COLOR, stroke_width=0.05,
fill='none'))

d2.save_svg("ring_polar_curve.svg")
svg_to_fixed_width_html_image(SVG('ring_polar_curve.svg').data.encode(
'ascii'), width="100%")

[2.80913411 2.80913411 2.80913411 2.80913411 2.80913411 2.80913411
2.80913411 2.80913411 2.80913411 2.80913411]
[-3.49019043 -3.5438993 -3.59760817 -3.65131705 -3.70502592 -
3.75873479
-3.81244366 -3.86615253 -3.9198614 -3.97357028]
```

[3.68503113 3.68503113 3.68503113 3.68503113 3.68503113 3.68503113
3.68503113 3.68503113 3.68503113 3.68503113]
[-3.57765385 -3.61736155 -3.65706925 -3.69677694 -3.73648464 -
3.77619233
-3.81590003 -3.85560772 -3.89531542 -3.93502311]
[3.92721447 3.92721447 3.92721447 3.92721447 3.92721447 3.92721447
3.92721447 3.92721447 3.92721447 3.92721447]
[-0.65184083 -0.68239082 -0.71294081 -0.7434908 -0.77404078 -
0.80459077
-0.83514076 -0.86569075 -0.89624073 -0.92679072]
[2.96480409 2.96480409 2.96480409 2.96480409 2.96480409 2.96480409
2.96480409 2.96480409 2.96480409 2.96480409]
[-0.69731031 -0.72924836 -0.76118641 -0.79312446 -0.82506251 -
0.85700056
-0.88893861 -0.92087665 -0.9528147 -0.98475275]
[2.27824741 2.27824741 2.27824741 2.27824741 2.27824741 2.27824741
2.27824741 2.27824741 2.27824741 2.27824741]
[2.35457003 2.32853457 2.30249912 2.27646366 2.25042821 2.22439275
2.1983573 2.17232184 2.14628639 2.12025093]
[3.52619721 3.52619721 3.52619721 3.52619721 3.52619721 3.52619721
3.52619721 3.52619721 3.52619721 3.52619721]
[1.81045973 1.77967968 1.74889963 1.71811958 1.68733953 1.65655948
1.62577943 1.59499938 1.56421933 1.53343928]
[2.38645963 2.38645963 2.38645963 2.38645963 2.38645963 2.38645963
2.38645963 2.38645963 2.38645963 2.38645963]
[3.74294693 3.70594664 3.66894634 3.63194605 3.59494575 3.55794546
3.52094516 3.48394487 3.44694457 3.40994428]
[2.24296215 2.24296215 2.24296215 2.24296215 2.24296215 2.24296215
2.24296215 2.24296215 2.24296215 2.24296215]
[4.78876321 4.73536507 4.68196692 4.62856877 4.57517062 4.52177247
4.46837432 4.41497617 4.36157802 4.30817988]
[4.27278407 4.27278407 4.27278407 4.27278407 4.27278407 4.27278407
4.27278407 4.27278407 4.27278407 4.27278407]
[-4.50707434 -4.54438516 -4.58169597 -4.61900679 -4.6563176 -
4.69362842
-4.73093923 -4.76825005 -4.80556086 -4.84287168]
[4.12516179 4.12516179 4.12516179 4.12516179 4.12516179 4.12516179
4.12516179 4.12516179 4.12516179 4.12516179]
[-4.12280691 -4.17154731 -4.2202877 -4.26902809 -4.31776849 -
4.36650888
-4.41524927 -4.46398967 -4.51273006 -4.56147045]
[4.52930479 4.52930479 4.52930479 4.52930479 4.52930479 4.52930479
4.52930479 4.52930479 4.52930479 4.52930479]
[-4.64557098 -4.68843558 -4.73130018 -4.77416477 -4.81702937 -
4.85989397
-4.90275857 -4.94562317 -4.98848777 -5.03135236]
[4.45860821 4.45860821 4.45860821 4.45860821 4.45860821 4.45860821
4.45860821 4.45860821 4.45860821 4.45860821]
[-2.90857965 -2.94729572 -2.9860118 -3.02472787 -3.06344395 -

3.10216003
-3.1408761 -3.17959218 -3.21830825 -3.25702433]
[5.69436584 5.69436584 5.69436584 5.69436584 5.69436584 5.69436584
5.69436584 5.69436584 5.69436584 5.69436584]
[-2.11010256 -2.15993307 -2.20976358 -2.25959409 -2.30942461 -
2.35925512
-2.40908563 -2.45891614 -2.50874665 -2.55857716]
[5.10253661 5.10253661 5.10253661 5.10253661 5.10253661 5.10253661
5.10253661 5.10253661 5.10253661 5.10253661]
[-0.49757741 -0.52219213 -0.54680684 -0.57142156 -0.59603627 -
0.62065098
-0.6452657 -0.66988041 -0.69449512 -0.71910984]
[5.82100493 5.82100493 5.82100493 5.82100493 5.82100493 5.82100493
5.82100493 5.82100493 5.82100493 5.82100493]
[-0.76514766 -0.79576506 -0.82638247 -0.85699988 -0.88761729 -
0.9182347
-0.9488521 -0.97946951 -1.01008692 -1.04070433]
[5.31181599 5.31181599 5.31181599 5.31181599 5.31181599 5.31181599
5.31181599 5.31181599 5.31181599 5.31181599]
[-0.03639041 -0.0616242 -0.086858 -0.11209179 -0.13732559 -
0.16255938
-0.18779317 -0.21302697 -0.23826076 -0.26349456]
[5.40402076 5.40402076 5.40402076 5.40402076 5.40402076 5.40402076
5.40402076 5.40402076 5.40402076 5.40402076]
[0.96083021 0.93851336 0.91619651 0.89387965 0.8715628 0.84924595
0.8269291 0.80461224 0.78229539 0.75997854]
[5.87071017 5.87071017 5.87071017 5.87071017 5.87071017 5.87071017
5.87071017 5.87071017 5.87071017 5.87071017]
[0.73457234 0.69534293 0.65611351 0.6168841 0.57765468 0.53842527
0.49919585 0.45996644 0.42073702 0.38150761]
[5.40707565 5.40707565 5.40707565 5.40707565 5.40707565 5.40707565
5.40707565 5.40707565 5.40707565 5.40707565]
[1.18202459 1.13388858 1.08575257 1.03761656 0.98948055 0.94134454
0.89320853 0.84507252 0.79693651 0.7488005]
[4.88666304 4.88666304 4.88666304 4.88666304 4.88666304 4.88666304
4.88666304 4.88666304 4.88666304 4.88666304]
[1.86809491 1.82291319 1.77773147 1.73254975 1.68736803 1.64218631
1.59700459 1.55182287 1.50664115 1.46145943]
[4.06993013 4.06993013 4.06993013 4.06993013 4.06993013 4.06993013
4.06993013 4.06993013 4.06993013 4.06993013]
[2.55863346 2.52220072 2.48576798 2.44933525 2.41290251 2.37646977
2.34003704 2.3036043 2.26717156 2.23073882]
[4.04976452 4.04976452 4.04976452 4.04976452 4.04976452 4.04976452
4.04976452 4.04976452 4.04976452 4.04976452]
[3.48338332 3.44983559 3.41628787 3.38274015 3.34919242 3.3156447
3.28209698 3.24854926 3.21500153 3.18145381]
[5.67812454 5.67812454 5.67812454 5.67812454 5.67812454 5.67812454
5.67812454 5.67812454 5.67812454 5.67812454]
[4.82538224 4.79419308 4.76300393 4.73181477 4.70062561 4.66943645

4.63824729 4.60705813 4.57586897 4.54467981]
[5.92685787 5.92685787 5.92685787 5.92685787 5.92685787 5.92685787
5.92685787 5.92685787 5.92685787 5.92685787]
[3.81895656 3.78204303 3.74512949 3.70821596 3.67130243 3.6343889
3.59747537 3.56056184 3.52364831 3.48673478]
[6.361871 6.361871 6.361871 6.361871 6.361871 6.361871 6.361871
6.361871
6.361871 6.361871]
[-4.15112942 -4.17823089 -4.20533237 -4.23243385 -4.25953532 -
4.2866368
-4.31373827 -4.34083975 -4.36794123 -4.3950427]
[7.68131027 7.68131027 7.68131027 7.68131027 7.68131027 7.68131027
7.68131027 7.68131027 7.68131027 7.68131027]
[-4.18163472 -4.2221703 -4.26270589 -4.30324148 -4.34377707 -
4.38431266
-4.42484825 -4.46538384 -4.50591942 -4.54645501]
[7.33896587 7.33896587 7.33896587 7.33896587 7.33896587 7.33896587
7.33896587 7.33896587 7.33896587 7.33896587]
[-2.86108796 -2.90223526 -2.94338256 -2.98452985 -3.02567715 -
3.06682444
-3.10797174 -3.14911904 -3.19026633 -3.23141363]
[6.35223673 6.35223673 6.35223673 6.35223673 6.35223673 6.35223673
6.35223673 6.35223673 6.35223673 6.35223673]
[-3.53541735 -3.56137661 -3.58733587 -3.61329513 -3.63925439 -
3.66521365
-3.69117291 -3.71713217 -3.74309143 -3.76905069]
[6.49527914 6.49527914 6.49527914 6.49527914 6.49527914 6.49527914
6.49527914 6.49527914 6.49527914 6.49527914]
[-4.57555581 -4.60317659 -4.63079736 -4.65841813 -4.6860389 -
4.71365968
-4.74128045 -4.76890122 -4.79652199 -4.82414276]
[6.87594011 6.87594011 6.87594011 6.87594011 6.87594011 6.87594011
6.87594011 6.87594011 6.87594011 6.87594011]
[-4.53391356 -4.57315893 -4.61240429 -4.65164966 -4.69089502 -
4.73014039
-4.76938576 -4.80863112 -4.84787649 -4.88712186]
[6.26045632 6.26045632 6.26045632 6.26045632 6.26045632 6.26045632
6.26045632 6.26045632 6.26045632 6.26045632]
[-1.37747244 -1.41362793 -1.44978341 -1.4859389 -1.52209438 -
1.55824987
-1.59440535 -1.63056084 -1.66671632 -1.70287181]
[7.27553613 7.27553613 7.27553613 7.27553613 7.27553613 7.27553613
7.27553613 7.27553613 7.27553613 7.27553613]
[-1.18702415 -1.2374447 -1.28786525 -1.33828579 -1.38870634 -
1.43912689
-1.48954743 -1.53996798 -1.59038853 -1.64080907]
[7.80412084 7.80412084 7.80412084 7.80412084 7.80412084 7.80412084
7.80412084 7.80412084 7.80412084 7.80412084]
[-2.39777481 -2.44815752 -2.49854022 -2.54892293 -2.59930564 -

2.64968835
-2.70007106 -2.75045376 -2.80083647 -2.85121918]
[6.28843431 6.28843431 6.28843431 6.28843431 6.28843431 6.28843431
6.28843431 6.28843431 6.28843431 6.28843431]
[-0.6096128 -0.63975545 -0.6698981 -0.70004075 -0.73018339 -
0.76032604
-0.79046869 -0.82061134 -0.85075399 -0.88089664]
[7.71194413 7.71194413 7.71194413 7.71194413 7.71194413 7.71194413
7.71194413 7.71194413 7.71194413 7.71194413]
[-2.3685504 -2.42087529 -2.47320017 -2.52552505 -2.57784994 -
2.63017482
-2.6824997 -2.73482458 -2.78714947 -2.83947435]
[6.97535248 6.97535248 6.97535248 6.97535248 6.97535248 6.97535248
6.97535248 6.97535248 6.97535248 6.97535248]
[-0.09091806 -0.120072 -0.14922594 -0.17837988 -0.20753382 -
0.23668775
-0.26584169 -0.29499563 -0.32414957 -0.35330351]
[7.68275775 7.68275775 7.68275775 7.68275775 7.68275775 7.68275775
7.68275775 7.68275775 7.68275775 7.68275775]
[2.06703537 2.01650412 1.96597287 1.91544163 1.86491038 1.81437913
1.76384788 1.71331664 1.66278539 1.61225414]
[7.12539841 7.12539841 7.12539841 7.12539841 7.12539841 7.12539841
7.12539841 7.12539841 7.12539841 7.12539841]
[0.2070786 0.17546009 0.14384157 0.11222305 0.08060453
0.04898602
0.0173675 -0.01425102 -0.04586954 -0.07748806]
[6.89514033 6.89514033 6.89514033 6.89514033 6.89514033 6.89514033
6.89514033 6.89514033 6.89514033 6.89514033]
[1.63247471 1.58703965 1.54160459 1.49616953 1.45073447 1.40529941
1.35986435 1.31442928 1.26899422 1.22355916]
[7.03309289 7.03309289 7.03309289 7.03309289 7.03309289 7.03309289
7.03309289 7.03309289 7.03309289 7.03309289]
[0.61311811 0.56697834 0.52083856 0.47469879 0.42855902 0.38241924
0.33627947 0.2901397 0.24399992 0.19786015]
[7.81443439 7.81443439 7.81443439 7.81443439 7.81443439 7.81443439
7.81443439 7.81443439 7.81443439 7.81443439]
[1.62482906 1.5966626 1.56849614 1.54032968 1.51216322 1.48399676
1.4558303 1.42766384 1.39949738 1.37133093]
[7.62973433 7.62973433 7.62973433 7.62973433 7.62973433 7.62973433
7.62973433 7.62973433 7.62973433 7.62973433]
[0.43705462 0.38640623 0.33575784 0.28510946 0.23446107
0.18381268
0.13316429 0.08251591 0.03186752 -0.01878087]
[7.8326684 7.8326684 7.8326684 7.8326684 7.8326684 7.8326684 7.8326684
7.8326684 7.8326684 7.8326684]
[3.31832629 3.27794568 3.23756506 3.19718444 3.15680383 3.11642321
3.07604259 3.03566198 2.99528136 2.95490074]
[7.75116603 7.75116603 7.75116603 7.75116603 7.75116603 7.75116603
7.75116603 7.75116603 7.75116603 7.75116603]

[3.42249927 3.39306872 3.36363817 3.33420762 3.30477707 3.27534653
3.24591598 3.21648543 3.18705488 3.15762433]
[7.09394707 7.09394707 7.09394707 7.09394707 7.09394707 7.09394707
7.09394707 7.09394707 7.09394707 7.09394707]
[3.87546223 3.85026297 3.82506371 3.79986444 3.77466518 3.74946592
3.72426666 3.69906739 3.67386813 3.64866887]
[6.39742866 6.39742866 6.39742866 6.39742866 6.39742866 6.39742866
6.39742866 6.39742866 6.39742866 6.39742866]
[2.72299888 2.66767122 2.61234357 2.55701591 2.50168825 2.44636059
2.39103294 2.33570528 2.28037762 2.22504996]
[7.39155722 7.39155722 7.39155722 7.39155722 7.39155722 7.39155722
7.39155722 7.39155722 7.39155722 7.39155722]
[2.70062316 2.65985141 2.61907965 2.5783079 2.53753614 2.49676438
2.45599263 2.41522087 2.37444911 2.33367736]
[7.81931357 7.81931357 7.81931357 7.81931357 7.81931357 7.81931357
7.81931357 7.81931357 7.81931357 7.81931357]
[4.0781656 4.0234417 3.96871781 3.91399391 3.85927002 3.80454612
3.74982223 3.69509834 3.64037444 3.58565055]
[8.51955003 8.51955003 8.51955003 8.51955003 8.51955003 8.51955003
8.51955003 8.51955003 8.51955003 8.51955003]
[-2.66185285 -2.69907283 -2.73629281 -2.77351279 -2.81073277 -
2.84795275
-2.88517273 -2.92239271 -2.95961269 -2.99683267]
[8.1235441 8.1235441 8.1235441 8.1235441 8.1235441 8.1235441 8.1235441
8.1235441 8.1235441 8.1235441]
[-3.48430187 -3.53738058 -3.59045929 -3.64353799 -3.6966167 -
3.74969541
-3.80277411 -3.85585282 -3.90893153 -3.96201023]
[9.89564803 9.89564803 9.89564803 9.89564803 9.89564803 9.89564803
9.89564803 9.89564803 9.89564803 9.89564803]
[-4.47000493 -4.49436617 -4.51872741 -4.54308865 -4.5674499 -
4.59181114
-4.61617238 -4.64053363 -4.66489487 -4.68925611]
[8.62949933 8.62949933 8.62949933 8.62949933 8.62949933 8.62949933
8.62949933 8.62949933 8.62949933 8.62949933]
[-3.03320697 -3.08235102 -3.13149507 -3.18063912 -3.22978317 -
3.27892722
-3.32807127 -3.37721532 -3.42635936 -3.47550341]
[9.54834343 9.54834343 9.54834343 9.54834343 9.54834343 9.54834343
9.54834343 9.54834343 9.54834343 9.54834343]
[-4.84431264 -4.88917995 -4.93404727 -4.97891459 -5.02378191 -
5.06864922
-5.11351654 -5.15838386 -5.20325118 -5.24811849]
[9.00235095 9.00235095 9.00235095 9.00235095 9.00235095 9.00235095
9.00235095 9.00235095 9.00235095 9.00235095]
[-3.50456948 -3.52895647 -3.55334347 -3.57773046 -3.60211746 -
3.62650445
-3.65089144 -3.67527844 -3.69966543 -3.72405243]
[9.72346315 9.72346315 9.72346315 9.72346315 9.72346315 9.72346315

9.72346315 9.72346315 9.72346315 9.72346315]
[-2.49181147 -2.53738007 -2.58294866 -2.62851726 -2.67408585 -
2.71965445
-2.76522305 -2.81079164 -2.85636024 -2.90192884]
[9.6951732 9.6951732 9.6951732 9.6951732 9.6951732 9.6951732 9.6951732
9.6951732 9.6951732 9.6951732]
[-0.49738962 -0.53519145 -0.57299328 -0.61079511 -0.64859694 -
0.68639877
-0.72420061 -0.76200244 -0.79980427 -0.8376061]
[8.35323206 8.35323206 8.35323206 8.35323206 8.35323206 8.35323206
8.35323206 8.35323206 8.35323206 8.35323206]
[-0.63640477 -0.65881136 -0.68121796 -0.70362455 -0.72603114 -
0.74843774
-0.77084433 -0.79325092 -0.81565751 -0.83806411]
[9.55304702 9.55304702 9.55304702 9.55304702 9.55304702 9.55304702
9.55304702 9.55304702 9.55304702 9.55304702]
[-0.30813028 -0.36011168 -0.41209307 -0.46407446 -0.51605585 -
0.56803724
-0.62001863 -0.67200002 -0.72398141 -0.7759628]
[9.27147057 9.27147057 9.27147057 9.27147057 9.27147057 9.27147057
9.27147057 9.27147057 9.27147057 9.27147057]
[-1.10439639 -1.14787889 -1.19136139 -1.2348439 -1.2783264 -
1.3218089
-1.36529141 -1.40877391 -1.45225641 -1.49573892]
[8.43932557 8.43932557 8.43932557 8.43932557 8.43932557 8.43932557
8.43932557 8.43932557 8.43932557 8.43932557]
[-2.26168734 -2.2906265 -2.31956566 -2.34850482 -2.37744398 -
2.40638314
-2.4353223 -2.46426146 -2.49320062 -2.52213978]
[9.41022508 9.41022508 9.41022508 9.41022508 9.41022508 9.41022508
9.41022508 9.41022508 9.41022508 9.41022508]
[0.67587743 0.6314972 0.58711697 0.54273674 0.49835651 0.45397628
0.40959605 0.36521582 0.32083559 0.27645536]
[9.50685045 9.50685045 9.50685045 9.50685045 9.50685045 9.50685045
9.50685045 9.50685045 9.50685045 9.50685045]
[2.38052462 2.3492315 2.31793838 2.28664526 2.25535213 2.22405901
2.19276589 2.16147277 2.13017965 2.09888653]
[8.85102623 8.85102623 8.85102623 8.85102623 8.85102623 8.85102623
8.85102623 8.85102623 8.85102623 8.85102623]
[2.3678801 2.32573272 2.28358535 2.24143798 2.19929061 2.15714324
2.11499587 2.0728485 2.03070112 1.98855375]
[8.2761851 8.2761851 8.2761851 8.2761851 8.2761851 8.2761851 8.2761851
8.2761851 8.2761851 8.2761851]
[0.34587894 0.29680049 0.24772204 0.19864359 0.14956513
0.10048668
0.05140823 0.00232978 -0.04674867 -0.09582713]
[8.90844727 8.90844727 8.90844727 8.90844727 8.90844727 8.90844727
8.90844727 8.90844727 8.90844727 8.90844727]
[0.97531885 0.95133793 0.927357 0.90337608 0.87939516 0.85541424

```

0.83143332 0.80745239 0.78347147 0.75949055]
[8.93240085 8.93240085 8.93240085 8.93240085 8.93240085 8.93240085
8.93240085 8.93240085 8.93240085 8.93240085]
[2.35997764 2.33331102 2.3066444 2.27997778 2.25331117 2.22664455
2.19997793 2.17331131 2.1466447 2.11997808]
[8.1659974 8.1659974 8.1659974 8.1659974 8.1659974 8.1659974 8.1659974
8.1659974 8.1659974 8.1659974]
[3.4029573 3.36766297 3.33236864 3.29707431 3.26177997 3.22648564
3.19119131 3.15589698 3.12060265 3.08530831]
[8.61943669 8.61943669 8.61943669 8.61943669 8.61943669 8.61943669
8.61943669 8.61943669 8.61943669 8.61943669]
[3.03983232 3.0012742 2.96271607 2.92415795 2.88559983 2.84704171
2.80848359 2.76992547 2.73136734 2.69280922]
[8.13999727 8.13999727 8.13999727 8.13999727 8.13999727 8.13999727
8.13999727 8.13999727 8.13999727 8.13999727]
[3.3702924 3.31884488 3.26739735 3.21594982 3.1645023 3.11305477
3.06160724 3.01015972 2.95871219 2.90726467]
[9.72619611 9.72619611 9.72619611 9.72619611 9.72619611 9.72619611
9.72619611 9.72619611 9.72619611 9.72619611]
[3.92031991 3.87566411 3.83100832 3.78635252 3.74169673 3.69704093
3.65238514 3.60772934 3.56307355 3.51841775]
[8.12996574 8.12996574 8.12996574 8.12996574 8.12996574 8.12996574
8.12996574 8.12996574 8.12996574 8.12996574]
[2.64228848 2.60934422 2.57639996 2.54345571 2.51051145 2.47756719
2.44462293 2.41167867 2.37873442 2.34579016]
[8.60312095 8.60312095 8.60312095 8.60312095 8.60312095 8.60312095
8.60312095 8.60312095 8.60312095 8.60312095]
[2.88656018 2.85802337 2.82948656 2.80094974 2.77241293 2.74387611
2.7153393 2.68680249 2.65826567 2.62972886]

```

<IPython.core.display.HTML object>

```

def cap_value(value, min_value, max_value):
    return max(min_value, min(value, max_value))

# Now use enlarged_dots with translate_to_polar_offset in the drawing
d2 = draw.Drawing(2 * R_OUTER + 2 * offset[0],
                 2 * R_OUTER + 2 * offset[1],
                 origin=(-R_OUTER - offset[0], -R_OUTER - offset[1]),
                 displayInline=False)
d2.set_pixel_scale(1)
r2 = draw.Circle(0, 0, r=R_OUTER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r2)
r3 = draw.Circle(0, 0, r=R_INNER, fill='none', stroke='red',
stroke_width="0.1")
d2.append(r3)

for x, y in enlarged_dots:

```

```
15) delta_y = random.uniform(DOT_DIAMETER_MM * 1, DOT_DIAMETER_MM *
15) delta_x = random.uniform(DOT_DIAMETER_MM * 1, DOT_DIAMETER_MM *

    polar_x, polar_y = translate_to_polar_offset(x, y, 5, R_INNER)
    polar_x2, polar_y2 = translate_to_polar_offset(x - delta_y, y, 5,
R_INNER)

    c = draw.Line(polar_x, polar_y, polar_x2, polar_y2,
stroke=ENGRAVE_COLOR, stroke_width="0.5")
    d2.append(c)

d2.save_svg("ring_digital_patch.svg")
svg_to_fixed_width_html_image(SVG('ring_digital_patch.svg').data.encode('ascii'), width="100%")
<IPython.core.display.HTML object>
```